Laboratório Funcional · Aula 12

Rômulo Amorim · RA 24000582-2 · Turma B
Gerado em 16/10/2026, 20:23:00 · tempo registrado no cronômetro: 58:07

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

Função agora(): É impura porque depende da data e hora atuais, pode retornar resultados diferentes mesmo sem receber parâmetros.
Função registra(msg): É impura porque utiliza print() para escrever uma mensagem na tela.

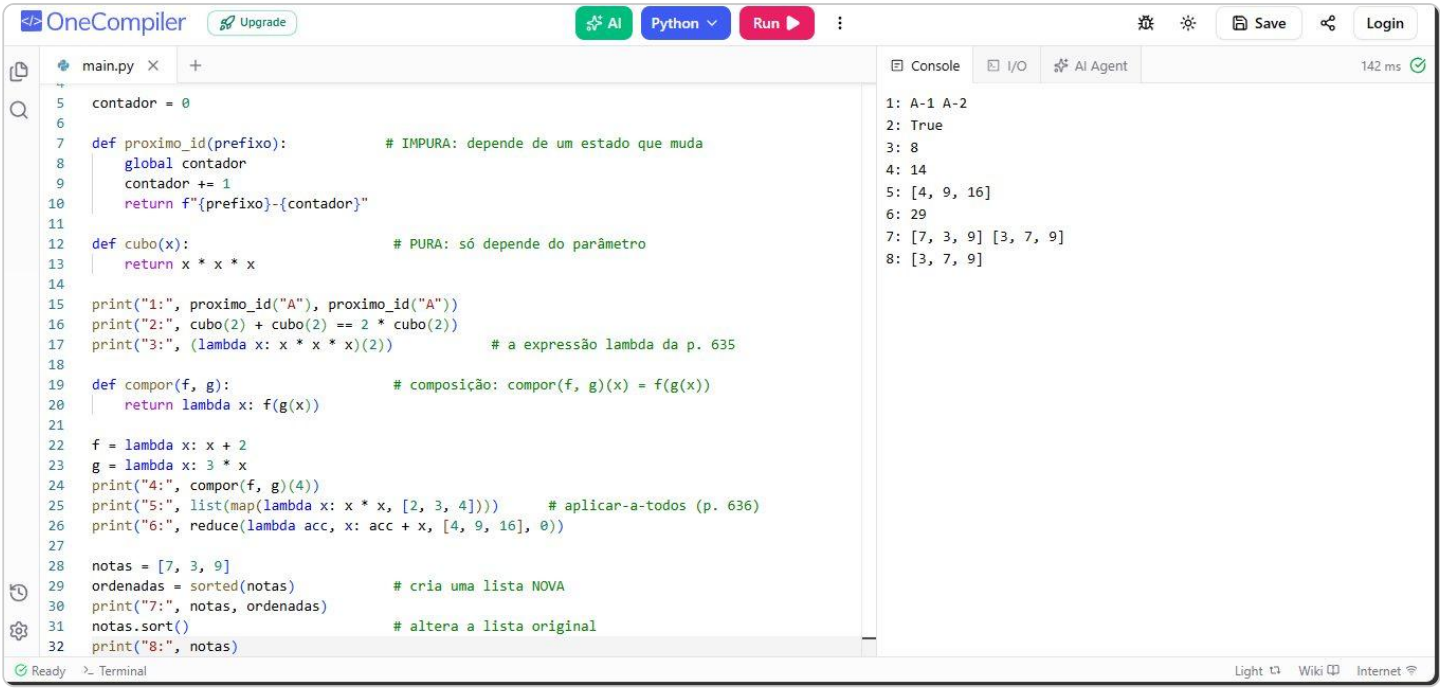
Prática 1: Funções puras e formas funcionais

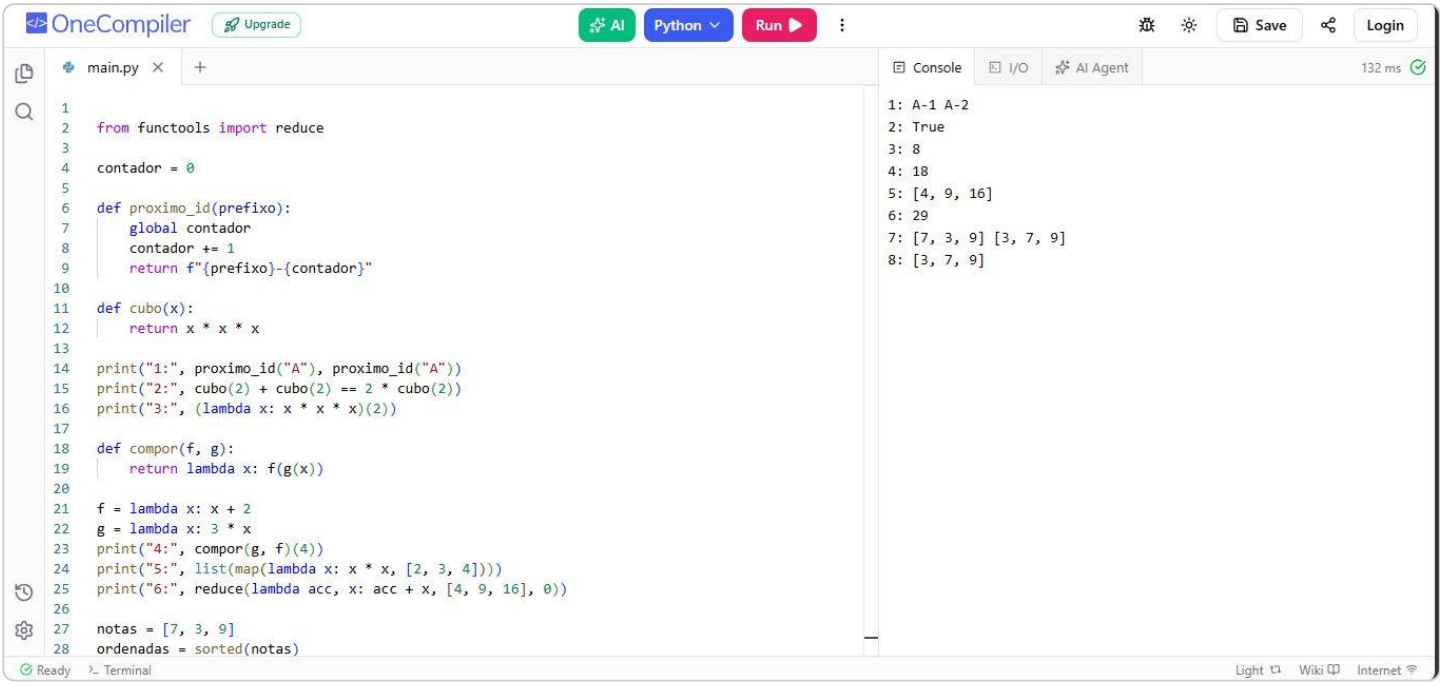
Previsão: O que as linhas 1, 4 e 7 vão imprimir?

linha 1 = A-1 A-2
linha 4 = 14
linha 7 = [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

não respondido





1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

Porque a função altera o contador a cada chamada, então retorna um número diferente. Isso dificulta os testes porque o resultado vai mudar mesmo usando a mesma entrada.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4): primeirp, g(4) = 3 x 4 = 12. depois, f(12) = 12 +2 = 14
compor(g, f)(4): primeiro, f(4)= 4 + 2 = 6. depois, g(6) = 3 x 6 = 18

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

O sorted(notas) cria uma nova lista ordenada, sem mexer a original e o notas.sort() ordena a própria lista. o sorted() segue mais o estilo funcional, porque não modifica a lista original.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

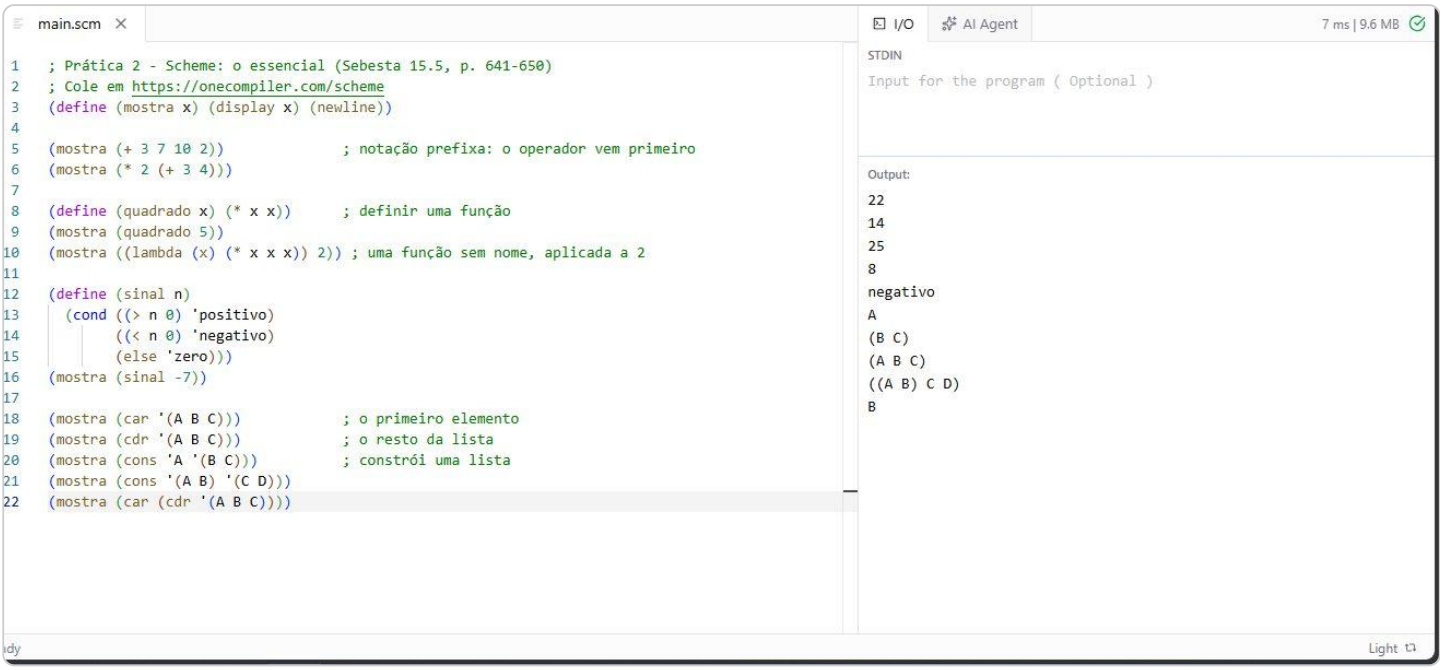
(B C)

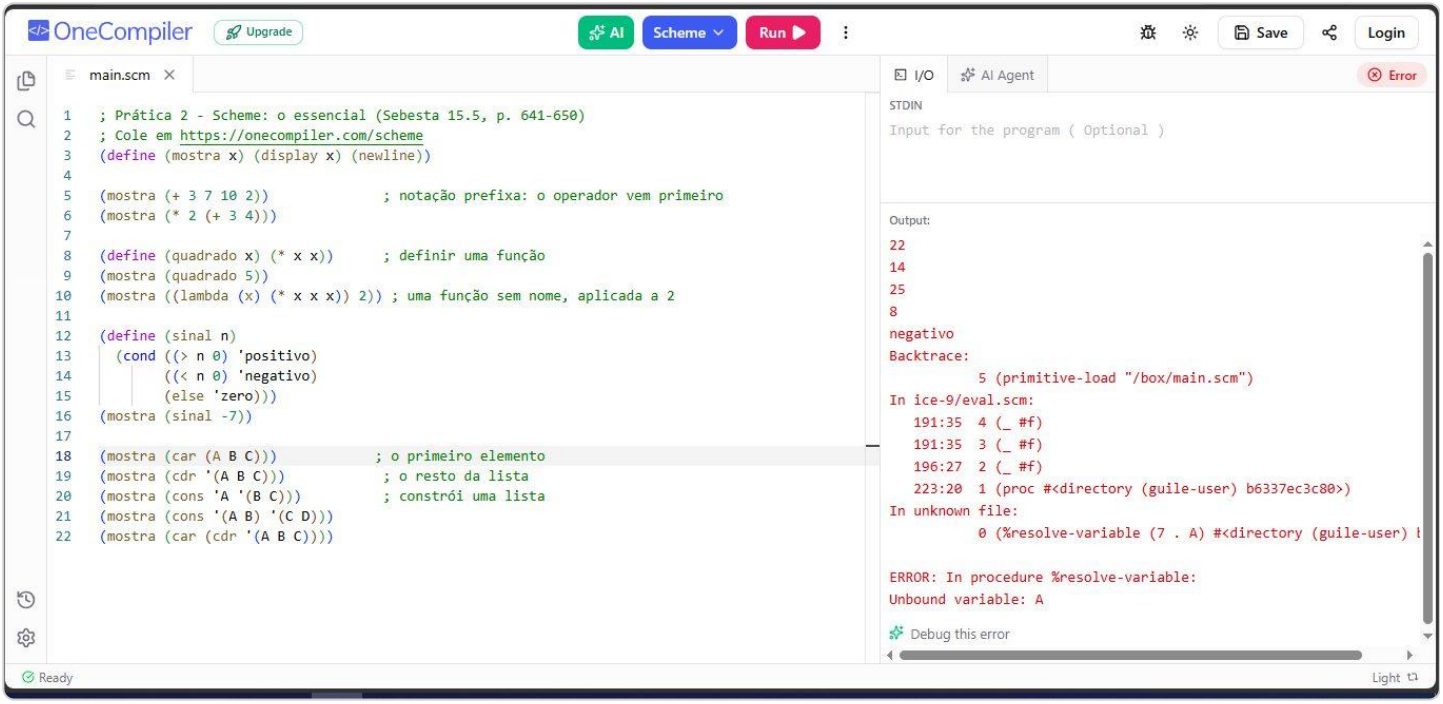
Previsão: (cons '(A B) '(C D))

((A B) (C D))

Saídas (original e sem o apóstrofo)

não respondido





2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo serve para tratar a lista como dado, sem executar. Sem ele, o Scheme tentou executar A como uma função e deu erro

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))
o resultado foi C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

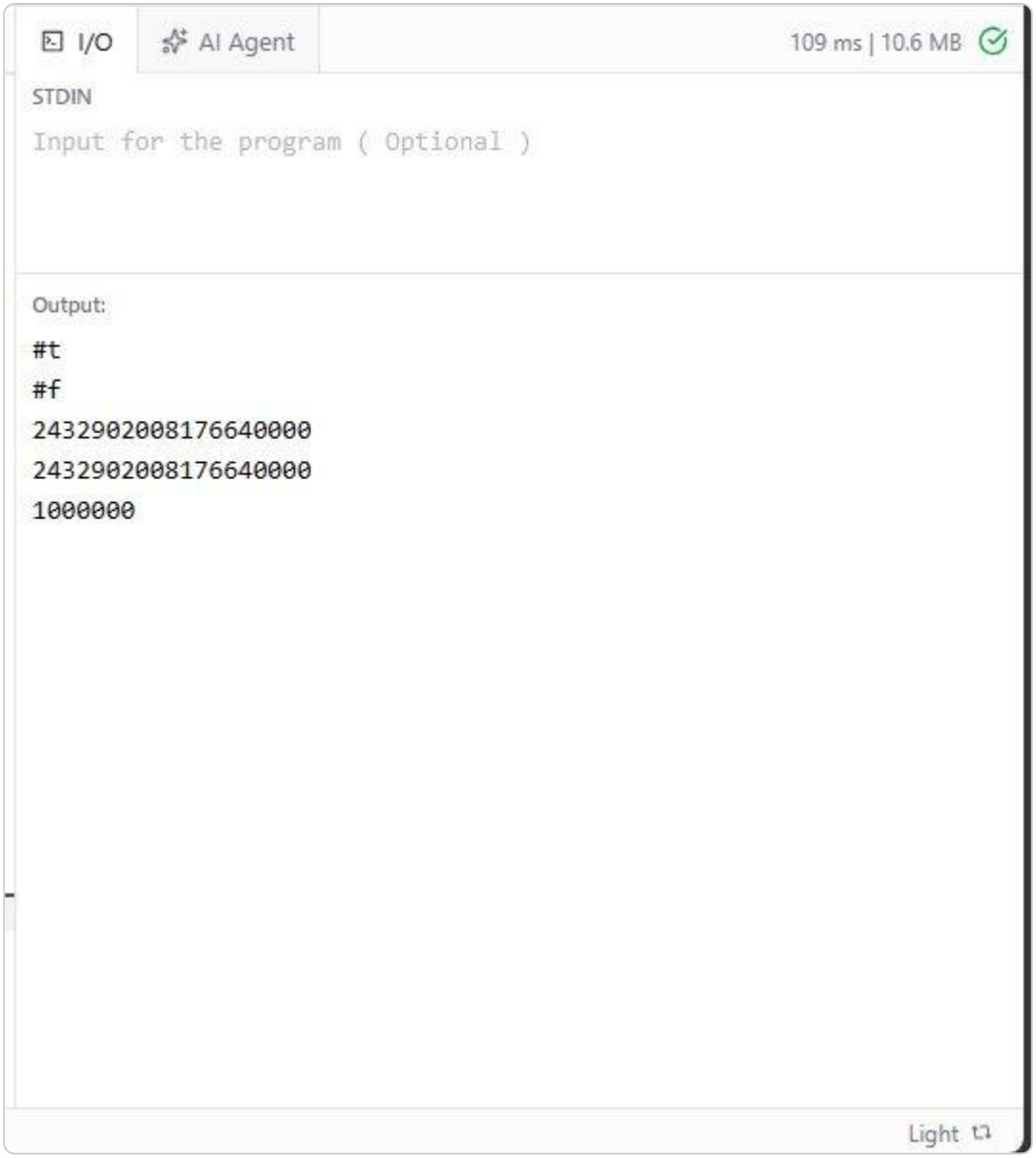
n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Em Scheme, depende, pode funcionar se houver otimização da recursão em cauda. Em Python, provavelmente vai dar erro por ultrapassar o limite de recursão.

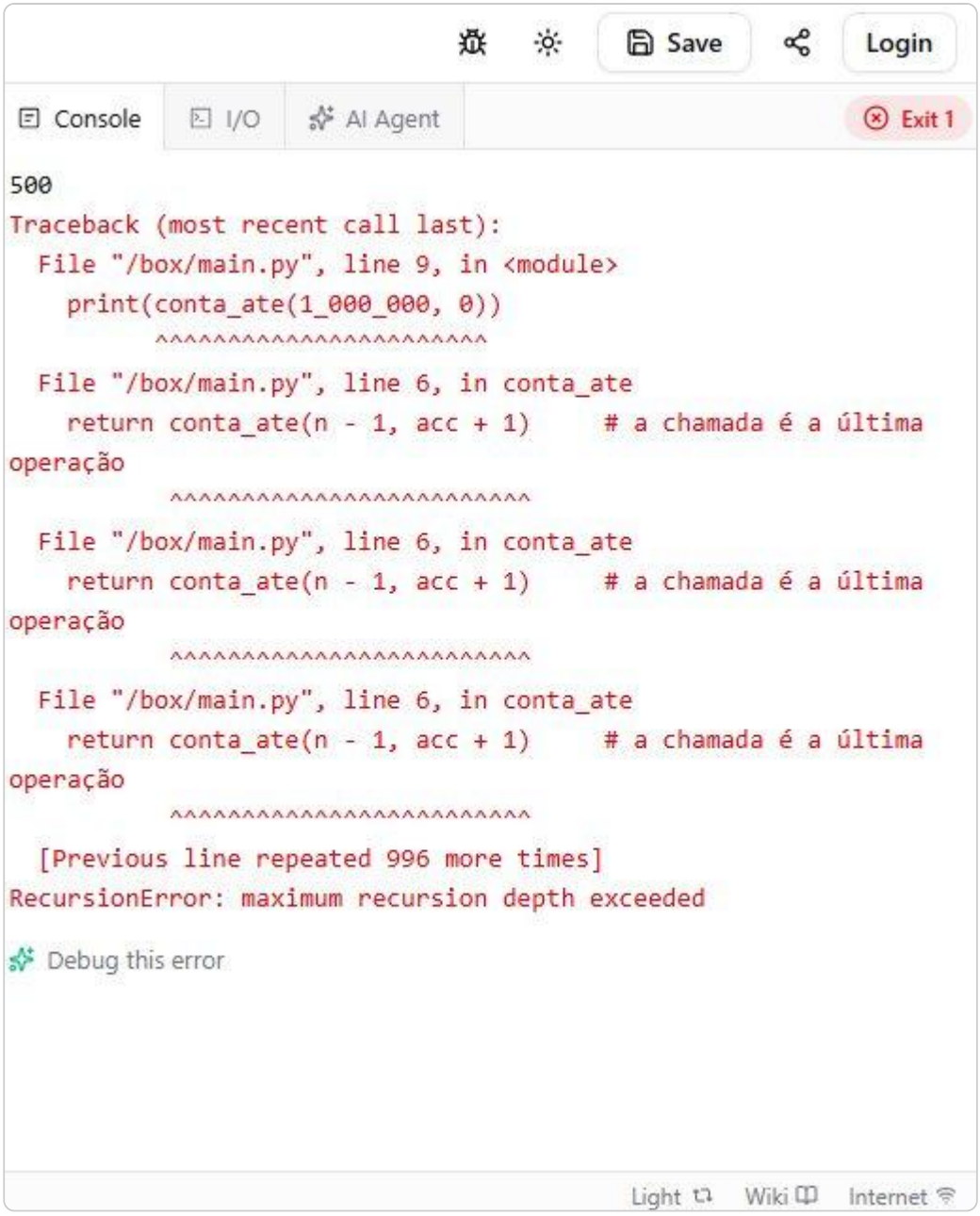
Saída do Scheme

não respondido



Saída do Python (as últimas linhas bastam)

não respondido



3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum cresce a quantidade de chamadas esperando para terminar. Na recursão em cauda isso não acontece porque a chamada recursiva é a última operação.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme otimiza a recursão em cauda e consegue reaproveitar a memória.E o Python tem um limite de chamadas recursivas e, ao ultrapassá-lo gera o erro RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

não respondido



4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
        (else
         (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi entender como a recursão funciona. Usei null para verificar se a lista está vazia, car para pegar o primeiro elemento e cdr para continuar com o restante da lista.

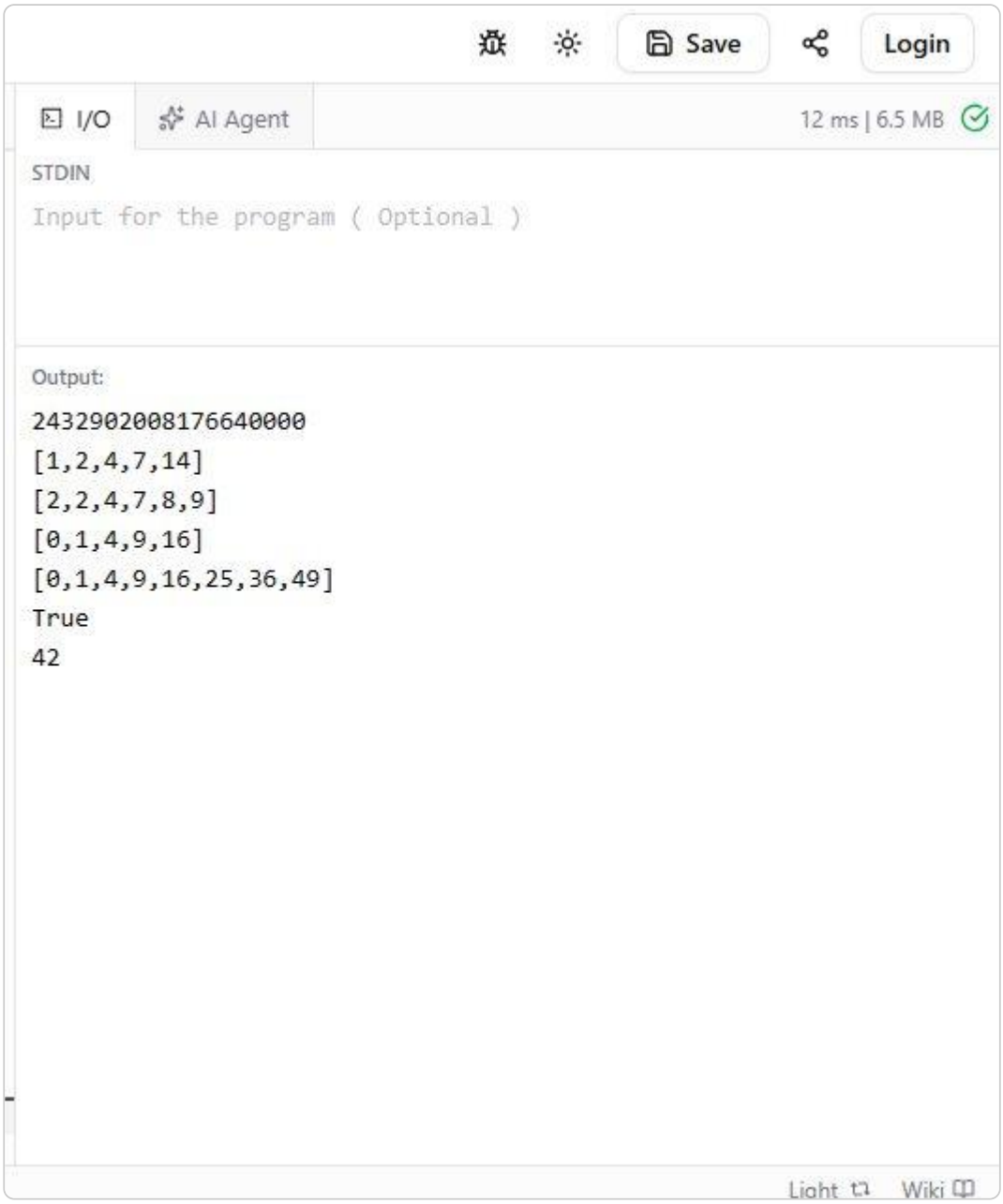
Prática 5: Haskell: padrões, compreensões e preguiça

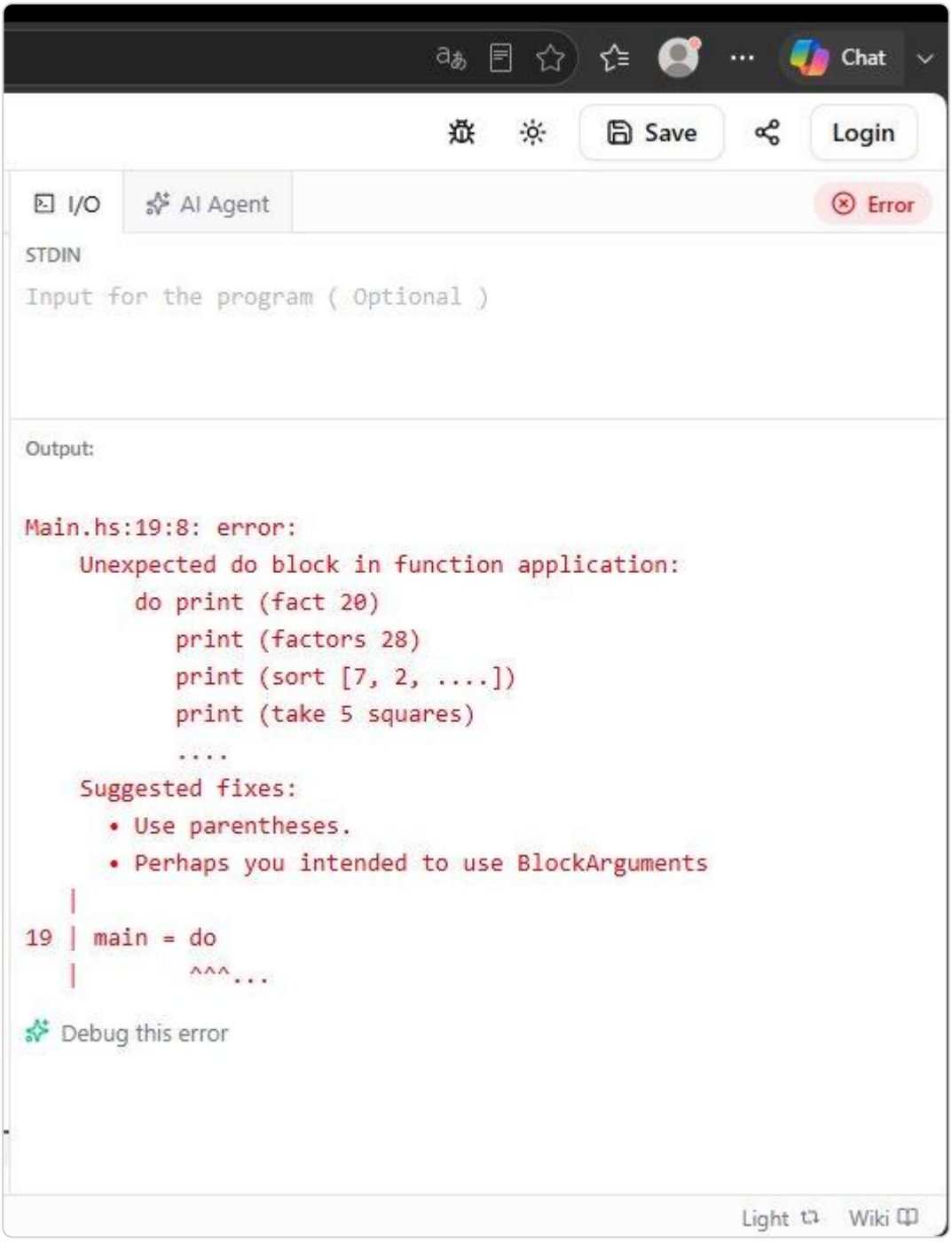
Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares - [0,1,4,9,16]
primeiro 42 (div 1 0) - 42

Saídas (original e com member 17)

não respondido





5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell usa a avaliação preguiçosa, calculando apenas os valores necessarios da lista infinita. Ja na função primeiro, o segundo parâmetro não é usado, então a divisão por zero não é executada.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

O member 16 squares terminou porque encontrou o número 16. Já o member 17 squares nunca termina porque a lista é infinita e o número 17 não aparece nela.
Eu corrigiria a função para parar a busca quando o elemento atual for maior que o número procurado.

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) -> 15

[1, 2, 3].map(vezes(5)) -> [5, 10, 15]

Saída do JavaScript

não respondido



Saída do Java

```
// Prática 6b - Lambdas e Streams em Java (Sebesta 15.10, p. 676)
// Cole em https://onecompiler.com/java
import java.util.*;
import java.util.function.*;

public class Main {
    record Produto(String nome, double preco) {}

    public static void main(String[] args) {
        Function<Integer, Integer> f = x -> x + 2;
        Function<Integer, Integer> g = x -> 3 * x;
        System.out.println(f.compose(g).apply(4));     // f(g(4))
        System.out.println(f.andThen(g).apply(4));     // g(f(4))

        List<Produto> produtos = List.of(new Produto("Teclado", 150), new Produto("Mouse",
80),
                                         new Produto("Monitor", 900), new Produto("Cabo",
35));
        double caros = produtos.stream()
            .filter(p -> p.preco() > 100)
            .mapToDouble(Produto::preco)
            .sum();
        System.out.println(caros);
        System.out.println(produtos.stream().map(p ->
p.nome().toUpperCase()).sorted().toList());
    }
}
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

A função devolvida lembra o valor 10 porque usa um closure, que guarda o valor da variavel do ambiente onde foi criada mesmo depois que somador(10) termina.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Os resultados foram diferentes porque a ordem de execução das funções muda. O compose executa primeiro g e depois f, enquanto o andThen executa primeiro f e depois g.
O f.compose(g) corresponde ao compor(f, g) da Prática 1, pois ambos calculam primeiro g(x) e depois f.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);

        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();

        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];

const aprovados = alunos
  .filter((aluno) ⇒ aluno.nota ⩾ 6)
  .map((aluno) ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

Saídas das suas versões

```
da 1 é 244
da 2 é ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional é mais fácil de ler porque fica mais curta e organizada. Já a versão com laço é mais fácil de depurar, pois permite acompanhar cada etapa e verificar os valores durante a execução.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais fácil de entender porque é mais curta e mostra diretamente o que o programa faz. Eu aceitaria esse custo em um sistema simples, mas não em um sistema que precisa de muita velocidade, como um sistema de tempo real.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi ver como é possível fazer as mesmas coisas com funções mais curtas, sem precisar de laços e variáveis acumuladoras.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro está em b <- h, que deveria ser b <= h. O compilador entende que b está sendo usado como uma lista, mas h é um único elemento. Corrigindo para b <= h, o código funciona.